

第9章 快剑如风

排序与二分

试炼场上，梁嘉峰演示一剑分两路之术："排序——让混乱变得有序。二分——在有序中快速定位。"

本章学习快速排序、归并排序和二分查找——这是算法世界的基石，掌握它们，你将拥有处理海量数据的能力。

JD103: 立桩定阵

AcWing 785 | JD103

试炼场上，弟子们按身高排成一列，但顺序完全乱了。梁嘉峰走过去，随便挑了一个弟子站到中间，然后让所有比他矮的站到左边，比他高的站到右边。

"现在左边和右边各自还是乱的，"赵晴儿看出来了。

"对。所以对左边和右边各自重复这个过程——挑一个人，矮的左、高的右。每次都能让一个人站到最终位置上，直到每一组只剩一个人。"梁嘉峰说，"这就是快速排序。一剑分两路，递归再出剑。"

给定一个长度为 n 的整数数列，请用快速排序将其从小到大排序。

输入格式

第一行包含整数 n 。第二行包含 n 个整数。

输出格式

一行， n 个排好序的整数，空格隔开。

输入样例

```
5
3 1 2 4 5
```

输出样例

```
1 2 3 4 5
```

解题思路：

选基准元素，双指针分区：左边 $\leq$ 基准，右边 $>$ 基准。递归处理左右两半。平均 $O(n\log n)$ 。

► 参考代码

JD104：照妖辨品

AcWing 786 | JD104

梁嘉峰指着那排弟子："我不需要所有人都排好——只要知道第 K 矮的人是谁。"

赵晴儿想了想："还是用刚才的方法：挑一个人站中间，矮的左、高的右。数一数左边有几个人——如果左边刚好有 $K-1$ 个人，那中间那个就是答案。如果左边人比 $K-1$ 多，说明答案在左边，只管左边就行。如果不够，就去右边找第 $K-左-1$ 矮的。"

"所以快排是把两边都排好，快选只需要管一边。"梁嘉峰点头，"省了一半的力气。"

给定一个长度为 n 的整数数列和一个整数 k ，求数列中第 k 小的数。

输入格式

第一行两个整数 n 和 k 。第二行 n 个整数。

输出格式

一行，第 k 小的数。

输入样例

```
5 3
3 1 2 4 5
```

输出样例

```
3
```

解题思路：

partition 后判断 k 在左半还是右半，只递归一半。平均 $O(n)$ 。

► 参考代码

JD105：双剑合阵

AcWing 787 | JD105

赵晴儿把弟子们两两分成一组，每组内部先按身高排好。然后两组合成一组四人——从两组的排头各看一眼，矮的先出列，依次接上。四人组再两两合并成八人组……

梁嘉峰看明白了："快排是先挑基准把人分开，归并是先把人拆到最小单位，再一层层合并上来。"

"对，"赵晴儿说，"合并时两边都是有序的，所以每次只需要比较排头， $O(n)$ 就能合并。拆到底再合上来，总共 $O(n\log n)$ 。而且最坏情况也一样——不像快排最坏会退化。"

给定一个长度为 n 的整数数列，请用归并排序将其从小到大排序。

输入格式

第一行包含整数 n 。第二行包含 n 个整数。

输出格式

一行， n 个排好序的整数，空格隔开。

输入样例

5

```
3 1 2 4 5
```

输出样例

```
1 2 3 4 5
```

解题思路：

递归拆分 → 合并两个有序数组。需要额外空间 $O(n)$ 。稳定排序。

► 参考代码

JD106：逆流之数

AcWing 788 | JD106

弟子们排成一列，每人身上有一个编号。赵晴儿发现，有些排在前面的人编号反而比后面的大——这就是“逆序对”，说明队伍很乱。

“逆序对越多，队伍越乱。”赵晴儿说，“但要一个个比太慢了。有没有 $O(n\log n)$ 的方法？”

李少白盯着归并排序看了一会儿，突然开窍：“归并的时候，左右两队都是从小到大排好的。从两队的排头开始比较——如果右边的先出列，说明右边这个人比左边队里剩下的所有人都小，那些都是逆序对！”

“没错。”赵晴儿笑了，“排完序，逆序对也数完了。”

给定一个长度为 n 的整数数列，求逆序对的数量。逆序对定义： $i < j$ 且 $a[i] > a[j]$ 。

输入格式

第一行包含整数 n 。第二行包含 n 个整数 ($1 \leq n \leq 100000$)。

输出格式

一行，逆序对的数量。

输入样例

```
5
2 3 1 5 4
```

输出样例

```
3
```

解题思路：

归并排序过程中，当右边元素先被选中时，左边剩余元素个数就是新增的逆序对数。用 `long long` 存结果。

► 参考代码

JD107：石壁听风

赵晴儿递给李少白一排刻着数字的石块，从左到右从小到大排好了。她问："数字 3 第一次出现在哪块石块上？最后一次出现在哪？"

李少白想从头数，但石块有十万块。赵晴儿拦住他："石块是有序的，不用一个个看。先看最中间那块——如果中间的数比 3 大，说明 3 只可能在左半边，右半边不用看了。每次都把范围砍一半，这就是二分。"

"找第一次出现的位置和最后一次出现的位置，各二分一次。"李少白接话道。

"对。如果整个范围里都找不到，就说明没有。"

给定一个升序排列的长度为 n 的整数数组和 q 个查询，对每个查询返回目标值的起始位置和终止位置（从 0 开始），不存在则返回 -1 -1 。

输入格式

第一行两个整数 n 和 q 。第二行 n 个整数（升序）。第三行 q 个待查询的整数。

输出格式

每行两个整数，起始位置和终止位置。

输入样例

```
6 3
1 2 2 3 3 4
2 3 5
```

输出样例

```
1 2
3 4
-1 -1
```

解题思路：

两次二分：找第一个 $\geq x$ 的位置（左边界），找最后一个 $\leq x$ 的位置（右边界）。

► 参考代码

JD108：剑指方根

AcWing 790 | JD108

赵晴儿在沙盘上写下一个数 n ："求它的三次方根。"

李少白试了几个整数，立方后不是太大就是太小。赵晴儿说："别猜了。三次方根一定在 -10000 到 10000 之间。你取中间值试试——如果中间值的立方比 n 大，说明真正的答案在左边；比 n 小，就在右边。范围不断缩小，直到精度足够。"

"和整数二分一样的思路，"李少白说，"只不过这次找的不是一个确切的位置，而是一个足够精确的浮点数。"

"对。整数二分停在某个整数上，浮点二分停在精度达标时。"

给定一个浮点数 n ，求它的三次方根。

输入格式

一个浮点数 n ($-10000 \leq n \leq 10000$)。

输出格式

n 的三次方根，保留6位小数。

输入样例

```
1000.000000
```

输出样例

```
10.000000
```

解题思路：

浮点二分：left = -10000, right = 10000, while(right - left > 1e-8), mid = (left + right) / 2。若 $\text{mid}^3 \geq n$, right = mid; 否则 left = mid。

► 参考代码